

Breadth First Search (BFS) : a way of visiting all vertices of a graph.

- start from a given vertex s
- BFS-tree is formed by edges taken during the alg.

BFS(V, E, s)

T = tree contains s

L = list containing s

while ($L \neq \emptyset$)

u = first element of L

 for every unvisited neighbor w of u

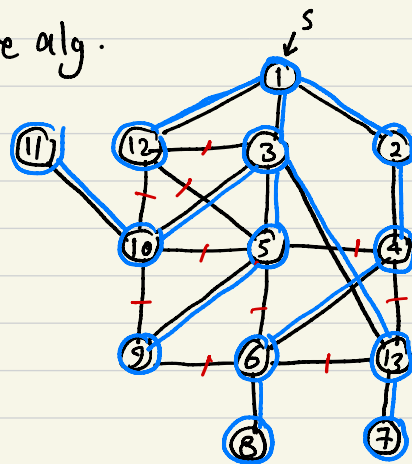
$T = T + \{(u, w)\}$

$L = L + \{w\}$

$L = L - \{u\}$

return T

T is called BFS-tree

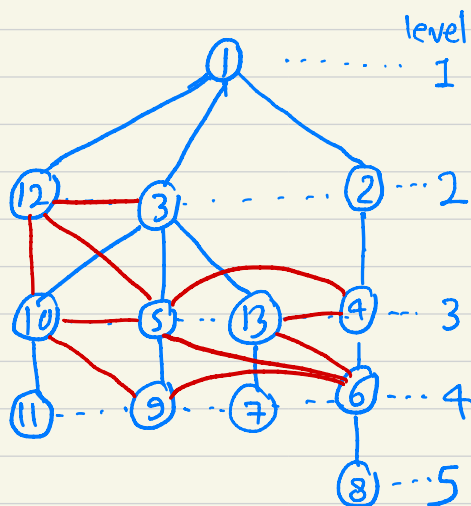


$L = 1, 2, 3, 12, 4, 5, 10, 13, 6, 9, 11, 7, 8$

blue edges: tree edges

red edges: non-tree edges.

two endpoints of every non-tree edge are either at the same level or between two consecutive levels. why?



Is G connected? Run BFS from an arbitrary vertex of G .

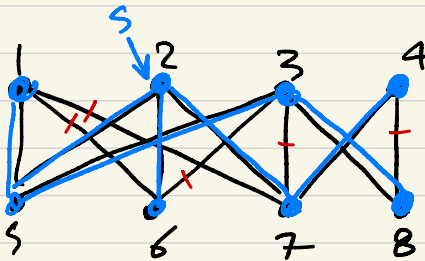
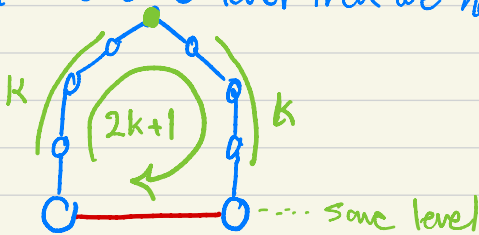
if BFS-tree has the same number of vertices as G ,
then G is connected, otherwise not.

Does G have a cycle? Run BFS

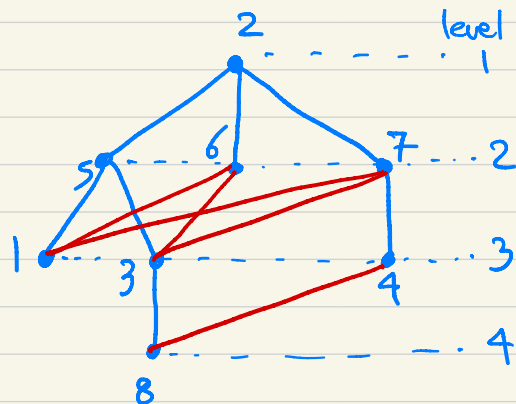
(Is G a tree?) if in the end we have a non-tree edge
then G has a cycle

(Every non-tree edge is part of a cycle)

Is G bipartite? Run BFS. If there is a non-tree edge with endpoints
at the same level then we have an odd cycle and G is not bipartite,
otherwise it is.



$L = \cancel{2}, \cancel{5}, \cancel{6}, \cancel{7}, \cancel{1}, \cancel{3}, \cancel{4}, \cancel{8}$



BFS: at each vertex visit all unvisited neighbors

DFS: " " " " one " " "

↳ if at some vertex you cannot proceed, go back to previous vertex and continue from there.

DFS-tree

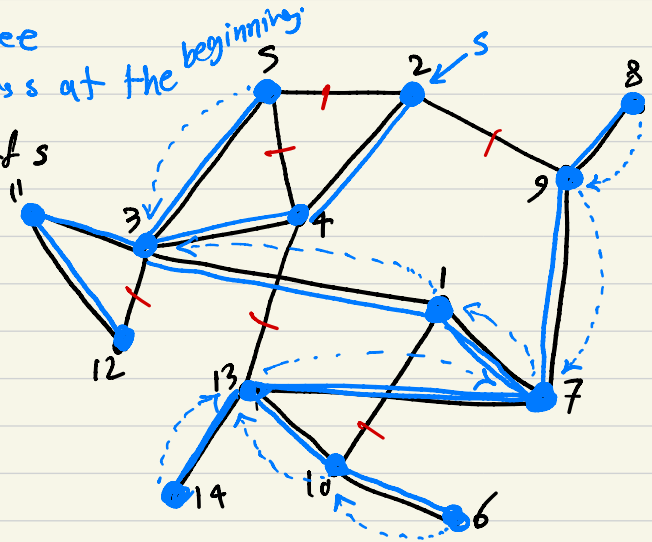
DFS (V, E, s, T) it contains s at the beginning

for each unvisited neighbor w of s

$T = T + \{(s, w)\}$

DFS (V, E, w, T)

return T



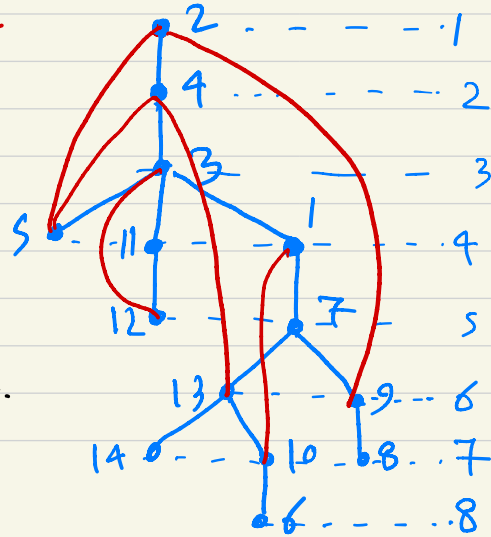
Order in which vertices are visited: 2, 4, 3, 1, 7, 9, 8, 13, 10, 6, 14, 5, 11, 12

T : DFS-tree

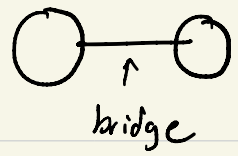
non-tree edges: backward edges

DFS: proposed by R. Tarjan

to determine if a graph is planar.



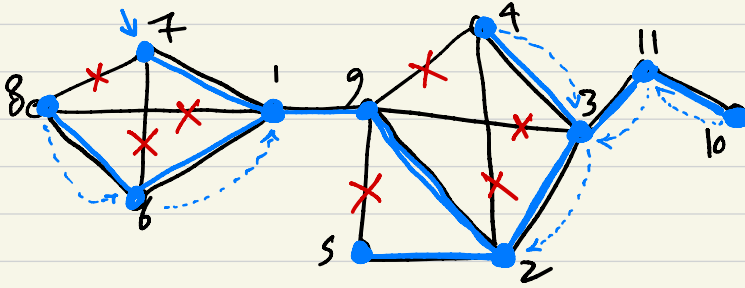
Identify all bridge in a connected graph G .



→ Run DFS on $G \rightarrow$ DFS tree

→ Each non-tree edge creates a cycle $\rightarrow$ mark all edges of cycle

→ all unmarked edges are bridges



Bridges (cut edges)

